LAPORAN PRAKTIKUM WEB SERVICE

SESI 11

DOKUMENTASI API OTOMATIS (OPENAPI/SWAGGER)



DISUSUN OLEH

Bambang Herlambang
NRP 5803024019

11 November 2026

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS KATOLIK WIDYA MANDALA
SURABAYA
2026

1 Tujuan dan ruang lingkup

Praktikum Sesi 11 menghasilkan dokumentasi OpenAPI dari anotasi kode pada layanan produk dan pesanan, menggabungkannya melalui gateway, serta menguji kesesuaian respons terhadap kontrak. Eksperimen drift memperlihatkan perbedaan antara field respons dan skema, kemudian memperbaikinya. Spesifikasi hasil gateway divalidasi dan diimpor ke Postman.

Pengerjaan menggunakan repository Lab 09 yang sama, melalui branch `lab11-openapi` yang melanjutkan branch Lab 10. Folder `catalog-service` berperan sebagai product-service. RabbitMQ dari Lab 10 tidak wajib dijalankan untuk Lab 11; peringatan broker tidak tersedia tidak menghalangi dokumentasi HTTP.

Repository pengumpulan:
`https://github.com/academicbambang/prakwebservice-lab09-microservices/tree/lab11-openapi`

2 Arsitektur dan implementasi dokumentasi

```
Browser --> gateway :8080/docs/
              GET /openapi.json (agregasi saat diminta)
                +--> catalog-service :3000/openapi.json
                +--> order-service   :3001/openapi.json
Kedua service juga menyediakan /docs/ masing-masing.
```

Anotasi YAML `@openapi` terdapat pada `catalog-service/app.js` dan `order-service/app.js`. Paket `swagger-jsdoc` membacanya melalui `apis: [__filename]` dengan `failOnErrors: true`; Swagger UI disajikan oleh setiap service. Perubahan anotasi memerlukan restart service.

Gateway mengambil spesifikasi kedua service secara paralel pada setiap permintaan `/openapi.json`, dengan timeout dua detik per sumber. Path diberi prefix `/product-service` dan `/order-service`; server gabungan memakai `/` agar Try it out mengarah ke origin gateway. Komponen dan referensi lokal diberi namespace `catalog_` dan `order_` untuk mencegah benturan nama.

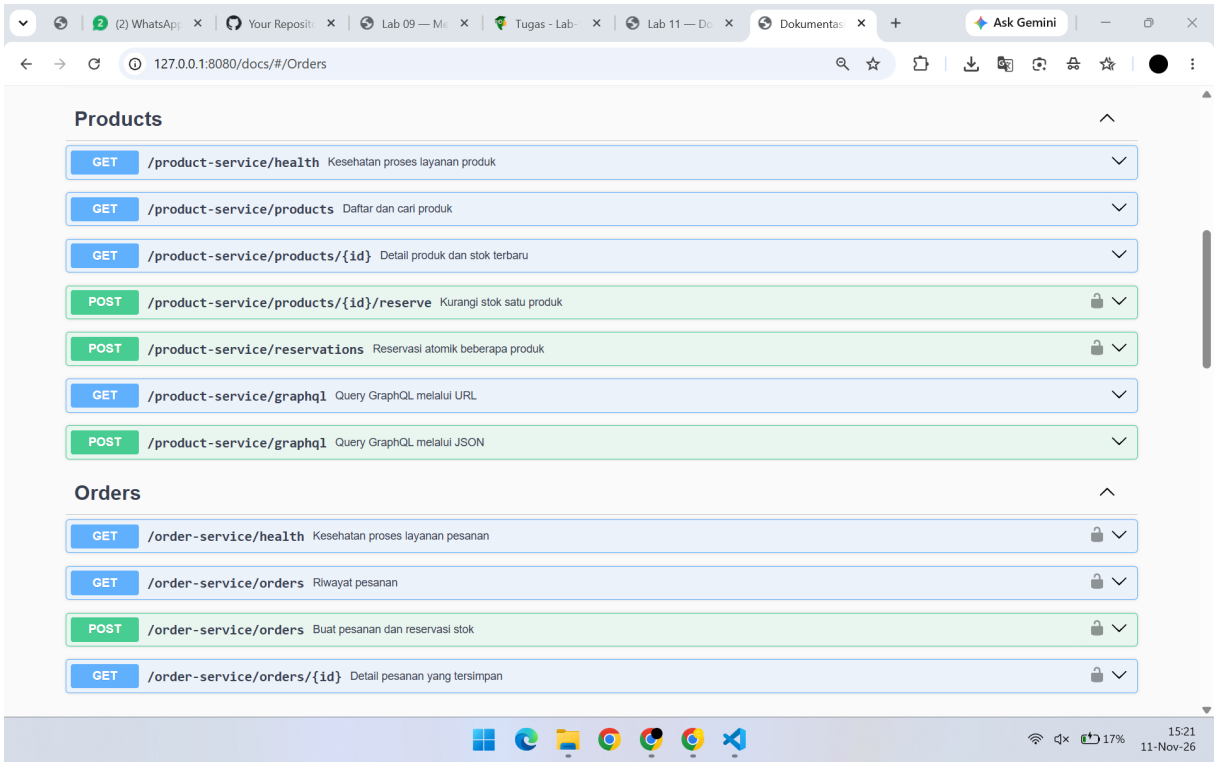
Rute dokumentasi dipasang sebelum proxy dan rate limiter. Kontrak gabungan memuat API key/401, kuota/429, dan kegagalan proxy/502. Jika satu sumber dokumentasi mati, sumber sehat tetap ditampilkan dan status sumber dilaporkan pada `x-documentation-sources`. Spesifikasi parsial tidak menyatakan seluruh sistem sehat.

Layanan	Spesifikasi	Dokumentasi interaktif
Katalog :3000	`/openapi.json`	`/docs/`
Pesanan :3001	`/openapi.json`	`/docs/`
Gateway :8080	`/openapi.json` gabungan	`/docs/` gabungan

Gambar 1 memperlihatkan Swagger UI gateway yang dibuka di `127.0.0.1:8080/docs/`. Halaman ini tidak berasal dari satu file anotasi, melainkan hasil agregasi spesifikasi kedua service saat diminta, sehingga gambar ini menjadi bukti utama bahwa dokumentasi tiap service berhasil digabung. Isinya dapat dibaca sebagai berikut.

- Grup **Products** memuat tujuh endpoint berawalan `/product-service`: `GET /health` (kesehatan proses layanan produk), `GET /products` (daftar dan cari produk), `GET /products/{id}` (detail produk dan stok terbaru), `POST /products/{id}/reserve` (kurangi stok satu produk), `POST /reservations` (reservasi atomik beberapa produk), serta `GET` dan `POST /graphql` (query GraphQL melalui URL dan melalui JSON).
- Grup **Orders** memuat empat endpoint berawalan `/order-service`: `GET /health`, `GET /orders` (riwayat pesanan), `POST /orders` (buat pesanan dan reservasi stok), dan `GET /orders/{id}` (detail pesanan yang tersimpan).
- Warna kotak mengikuti metode HTTP, yaitu biru untuk `GET` dan hijau untuk `POST`. Ikon gembok muncul pada `POST /products/{id}/reserve`, `POST /reservations`, dan seluruh endpoint `/order-service`, yang berarti endpoint tersebut meminta API key lewat tombol Authorize. Endpoint baca katalog (health, daftar, detail, dan GraphQL) tidak bergembok.

Awalan `/product-service` dan `/order-service` ditambahkan gateway saat menggabungkan spesifikasi, bukan ditulis pada anotasi service, dan alamat server gabungan adalah `/` sehingga Try it out memanggil origin gateway. Dengan demikian Gambar 1 memenuhi tujuan praktikum 1 (spesifikasi dan Swagger UI dihasilkan dari anotasi) dan tujuan 2 (dokumentasi beberapa service digabung pada satu halaman).



Gambar 1. Dua grup endpoint (Products dan Orders) pada dokumentasi gabungan gateway. Sumber: Screenshot (1964).png.

3 Cara menjalankan dan konfigurasi

Gunakan Node.js versi 22 atau lebih baru. Dari root repository:

```
npm ci
npm run setup
npm start
```

Launcher memulai katalog pada port 3000, pesanan pada 3001, dan gateway pada 8080. Setelah ketiganya siap, buka `http://127.0.0.1:8080/docs/`. Perintah setup membuat API key lokal pada `gateway/.env`; masukkan nilainya melalui tombol Authorize untuk pesanan dan reservasi stok. Nilai key tidak dimasukkan ke laporan atau repository.

Variabel	Kegunaan
PORT, HOST	Port gateway (default 8080) dan alamat bind.
CATALOG_PORT, ORDER_PORT	Port backend (default 3000 dan 3001).
CATALOG_SERVICE_URL	Origin katalog untuk gateway dan pemanggilan oleh order.
ORDER_SERVICE_URL	Origin layanan pesanan untuk gateway.
API_KEY, API_KEY_SHA256	Key atau alternatif hash key untuk autentikasi gateway.
TRUST_PROXY	Jumlah proxy tepercaya; default 0 untuk akses lokal.
PRODUCT_DOC_URL, ORDER_DOC_URL	Override URL penuh spesifikasi; default mengikuti origin service.
AMQP_URL, ORDER_EXCHANGE	Konfigurasi event Lab 10, opsional untuk Lab 11.

3.1 Contoh permintaan pesanan

Pada `POST /order-service/orders`, klik Try it out dan isi:

```
{"customerName":"Praktikum Lab 11",
 "items":[{"productId":"prod-001","quantity":2}]}
```

Respons sukses adalah 201. `GET /product-service/products/prod-001` dipakai untuk memeriksa stok. Data berada di memori dan kembali ke seed saat service di-restart.

3.2 Penyesuaian terhadap contoh panduan

ID produk pada repo berbentuk string `prod-001`, bukan integer 1. Body pesanan memakai `customerName` dan array `items`; jumlah setiap item bernama `quantity` dan dibatasi 1–100. Karena itu `quantity` 100 dipakai untuk menguji stok tidak cukup (409), sedangkan 999 menghasilkan input tidak valid (400). Respons menggunakan envelope `data`, dengan field `service` pada daftar. Field `category` sudah tersedia sejak Lab 09; eksperimen drift dilakukan dengan menghapus sementara anotasinya, bukan menambah field data baru.

4 Hasil pengujian dan eksperimen drift

4.1 Demo otomatis yang telah direkam

Demo membuka Swagger UI dan menggunakan Try it out/Execute. Rekaman asli tersedia pada `docs/evidence/lab11/results.json`; waktu eksekusi tercatat 11 November 2026 pukul 08:13 UTC (15:13 UTC+7).

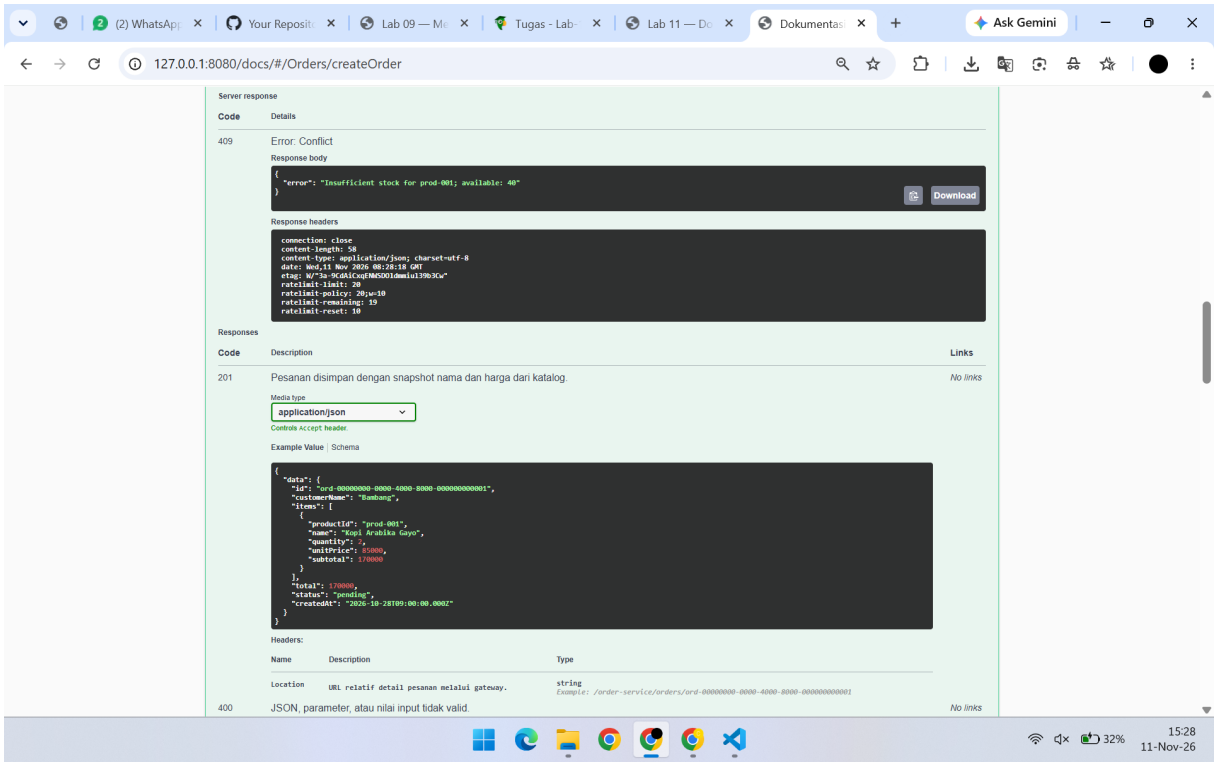
Skenario	HTTP	Hasil aktual
`prod-001`, quantity 2	201	Pesanan tersimpan; stok 40 menjadi 38.
`prod-001`, quantity 100	409	Stok tidak cukup; tersedia 38 setelah order sukses.
`prod-099`, quantity 2	404	Produk tidak ditemukan.
quantity 999	400	Melanggar batas input quantity 1–100.
Katalog dihentikan	503	Layanan pesanan melaporkan dependensi tidak tersedia.

Semua respons pesanan yang direkam lolos validasi skema AJV terhadap spesifikasi yang telah didereference. Spesifikasi lolos Swagger Parser. Verifikasi lokal menghasilkan 78 dari 78 tes lulus; CI pada pembaruan bukti manual juga sukses. Tiga tes regresi baru mencakup field produk yang belum didokumentasikan, benturan nama komponen/pengambilan ulang spesifikasi, serta sumber dokumentasi yang gagal.

4.2 Bukti manual respons 409

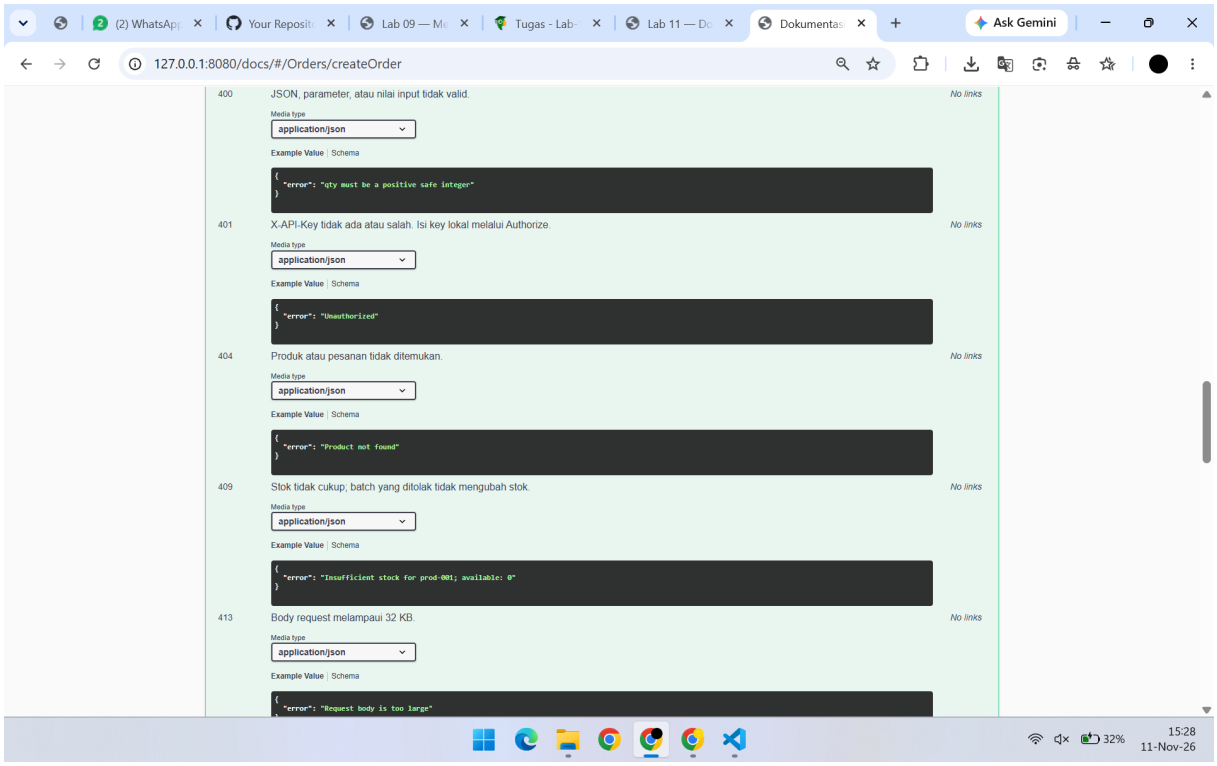
Skenario ini menguji janji kontrak untuk kasus stok tidak cukup (tujuan praktikum 3). Pada `POST /order-service/orders`, Try it out dijalankan dengan `quantity` 100 untuk `prod-001`, yang saat itu stoknya 40.

Gambar 2 diambil dari halaman `#/Orders/createOrder` dan terdiri dari dua bagian. Bagian atas adalah Server response: kode 409 dengan keterangan Error: Conflict, isi `{"error": "Insufficient stock for prod-001; available: 40"}`, dan header respons. Header itu memuat `content-type: application/json; charset=utf-8` serta `ratelimit-limit: 20`, `ratelimit-policy: 20;w=10`, `ratelimit-remaining: 19`, dan `ratelimit-reset: 10`, yang sesuai dengan kuota 20 request per 10 detik per IP pada deskripsi spesifikasi gateway. Bagian bawah adalah tabel Responses, yaitu kontrak yang didokumentasikan, dimulai dari 201 (pesanan disimpan dengan snapshot nama dan harga dari katalog). Contoh isi 201 berupa objek `data` dengan `id`, `customerName`, `items` (`productId`, `name`, `quantity`, `unitPrice`, `subtotal`), `total`, `status` bernilai `pending`, dan `createdAt`, ditambah header `Location` berisi URL relatif detail pesanan melalui gateway.



Gambar 2. Respons aktual 409 pada sesi manual (`quantity` 100), beserta awal tabel respons yang didokumentasikan. Sumber: Screenshot (1973).png.

Gambar 3 adalah bagian bawah tabel Responses untuk endpoint yang sama. Setiap baris memuat kode, deskripsi, dan contoh isi berbentuk `{"error": ...}`: 400 untuk JSON, parameter, atau nilai input tidak valid; 401 untuk X-API-Key yang tidak ada atau salah (pengguna diminta mengisi key lokal lewat Authorize); 404 untuk produk atau pesanan tidak ditemukan; 409 untuk stok tidak cukup, dengan catatan bahwa batch yang ditolak tidak mengubah stok; dan 413 untuk body request yang melampaui 32 KB. Baris 409 inilah yang dicocokkan dengan hasil pada Gambar 2. Kode dan maknanya sama, dan pesan aktual mengikuti pola contoh dokumentasi (`Insufficient stock for prod-001; available: ...`). Perbedaannya hanya pada angka, karena contoh dokumentasi menulis `available: 0` sebagai nilai statis, sedangkan respons aktual menunjukkan 40 sesuai stok saat itu. Karena angka pada contoh bukan hasil request, selisih ini bukan drift.



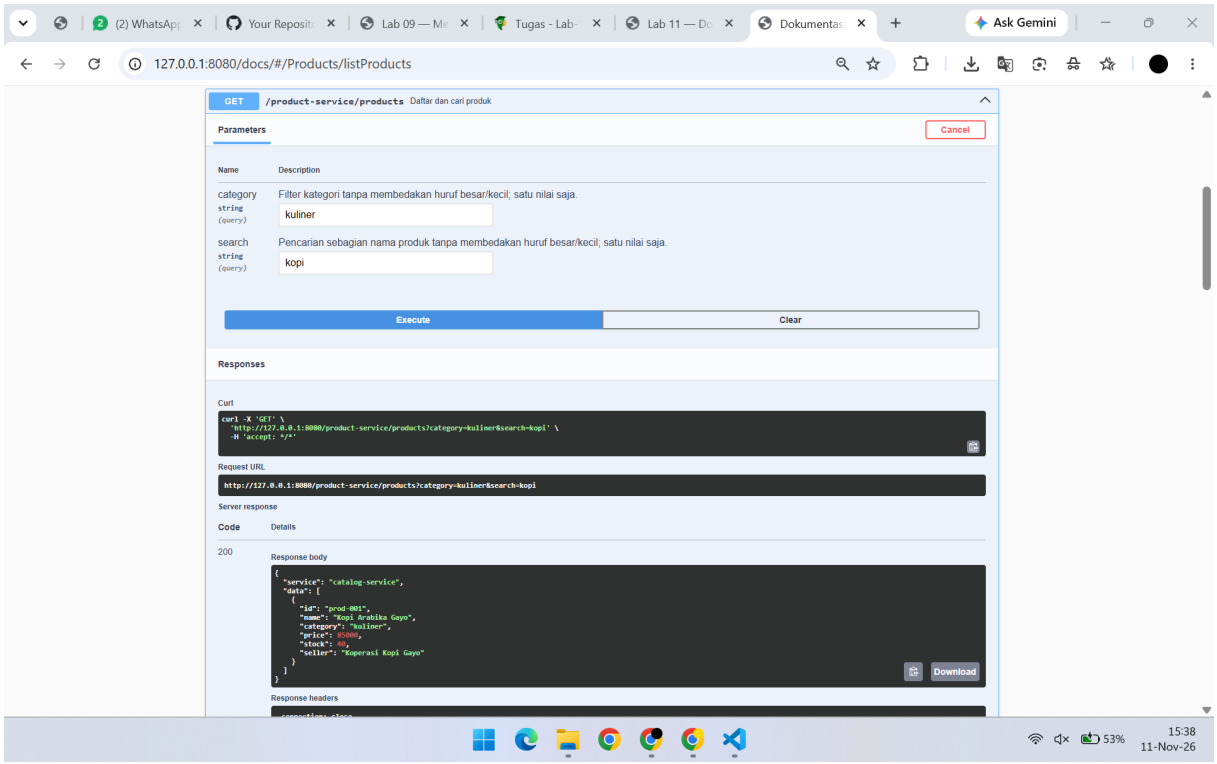
Gambar 3. Status 409 pada daftar respons yang didokumentasikan untuk endpoint yang sama. Sumber: Screenshot (1974).png.

Dari kedua gambar, status 409 yang muncul pada respons nyata memang tercantum di kontrak, sehingga respons aktual sesuai dengan dokumentasi. Angka 40 berasal dari sesi manual terpisah, sedangkan angka 38 pada tabel Bagian 4.1 berasal dari demo otomatis setelah order `quantity` 2. Contoh `available: 0` pada dokumentasi adalah contoh statis, bukan hasil request manual.

4.3 Drift sebelum dan sesudah diperbaiki

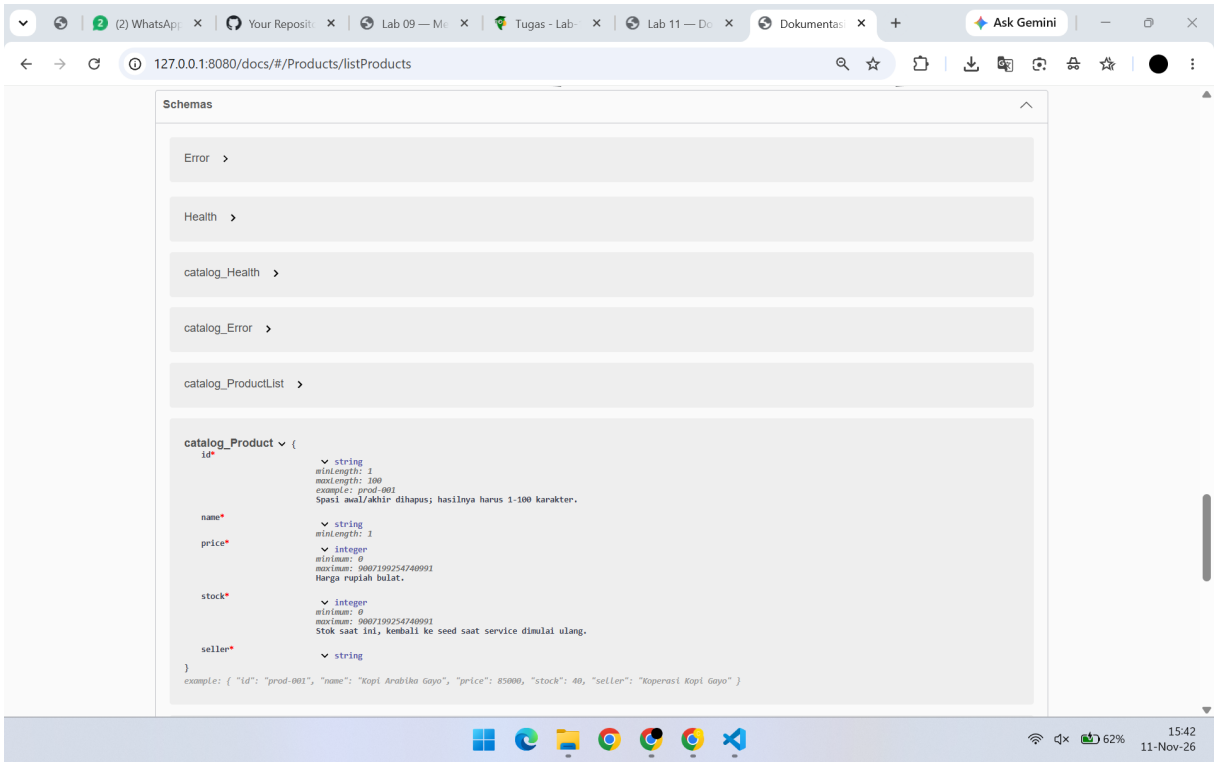
Eksperimen ini mengikuti tujuan praktikum 4. Pada skema Product dalam `catalog-service/app.js`, anotasi `category` dihapus sementara dari `required`, `properties`, dan `example`. Setelah file disimpan dan aplikasi di-restart, tiga tangkapan layar diambil berurutan: respons nyata, skema saat drift, lalu skema setelah diperbaiki.

Gambar 4 adalah respons nyata. Pada halaman `#/Products/listProducts`, endpoint `GET /product-service/products` dijalankan dengan parameter `category=kuliner` dan `search=kopi`. Keduanya terlihat pada kolom input, pada perintah curl yang dibuat Swagger UI, dan pada Request URL `http://127.0.0.1:8080/product-service/products?category=kuliner&search=kopi`, sehingga filter memang terkirim. Server menjawab 200 dengan `service: catalog-service` dan satu produk, yaitu `prod-001` (Kopi Arabika Gayo) dengan `category` kuliner, `price` 85000, `stock` 40, dan `seller` Koperasi Kopi Gayo. Artinya kode tetap mengirim `category` meskipun anotasinya sedang dihapus.



Gambar 4. Field `category` tetap ada pada respons nyata meskipun anotasinya dihapus sementara. Sumber: Screenshot (1978).png.

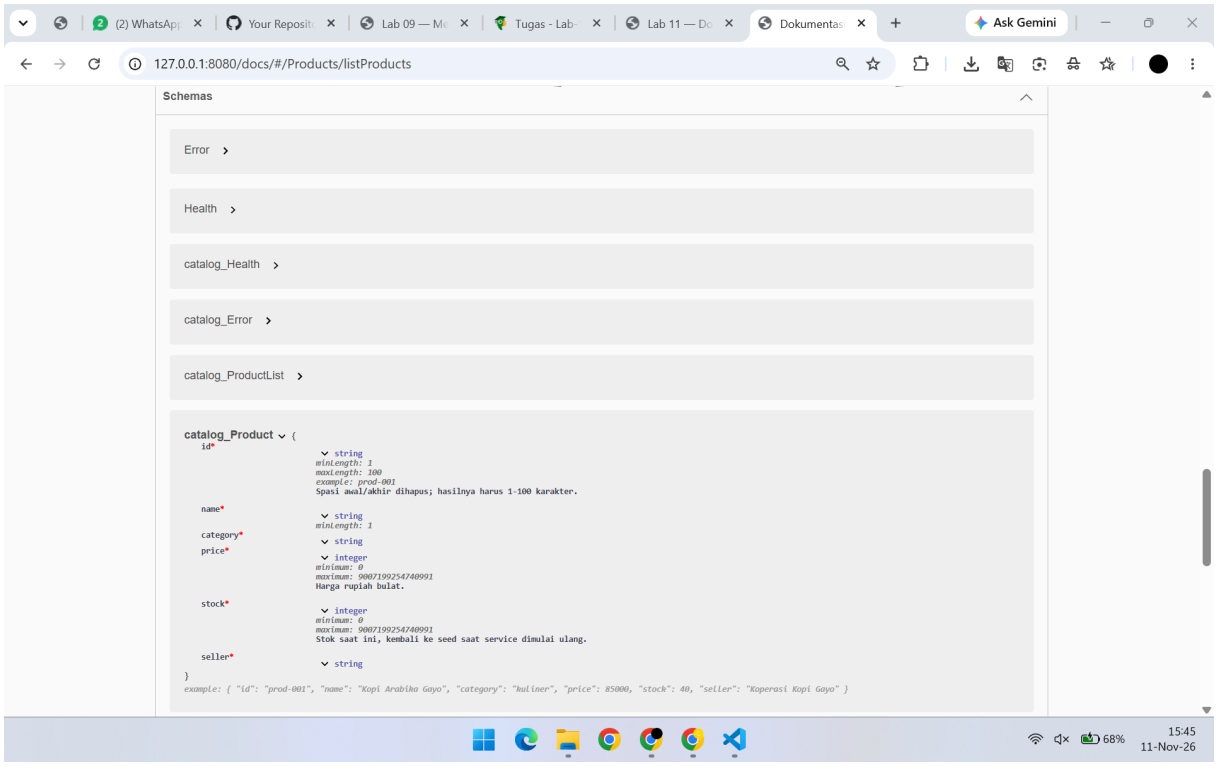
Gambar 5 memperlihatkan bagian Schemas pada halaman yang sama. Daftar skema berisi Error, Health, `catalog_Health`, `catalog_Error`, `catalog_ProductList`, dan `catalog_Product`; awalan `catalog_` berasal dari namespace yang diberikan gateway. Skema `catalog_Product` yang dibuka hanya memiliki lima field wajib (ditandai bintang merah), yaitu `id`, `name`, `price`, `stock`, dan `seller`, dan contoh di bagian bawah skema juga hanya memuat kelima field itu. Tidak ada `category`, padahal Gambar 4 membuktikan field tersebut dikirim. Inilah drift, yaitu dokumentasi menjanjikan bentuk respons yang sudah tidak sama dengan kenyataan.



Gambar 5. Skema `catalog_Product` saat terjadi drift (tanpa `category`). Sumber: Screenshot (1983).png.

Ketiga bagian anotasi kemudian dikembalikan, aplikasi di-restart, dan Swagger di-refresh. Gambar 6 menunjukkan skema yang sama setelah perbaikan. Perbedaannya hanya pada `category`: field ini muncul di antara `name` dan `price` dengan tipe string dan tanda wajib, dan contoh di bawah skema kembali memuat `"category": "kuliner"`. Field lain tidak berubah. Perbandingan kedua skema dirangkum berikut.

Field	Tipe dan batas (sama di kedua gambar)	Gambar 5	Gambar 6
`id`	string, `minLength` 1, `maxLength` 100, contoh `prod-001`	wajib	wajib
`name`	string, `minLength` 1	wajib	wajib
`category`	string	tidak ada	wajib
`price`	integer, `minimum` 0, `maximum` 9007199254740991	wajib	wajib
`stock`	integer, `minimum` 0, `maximum` 9007199254740991	wajib	wajib
`seller`	string	wajib	wajib
contoh skema	id, name, price, stock, seller	tanpa `category`	dengan `category`



Gambar 6. Skema `catalog_Product` setelah anotasi dipulihkan dan service di-restart. Sumber: Screenshot (1984).png.

Kode final sudah memiliki `property`, `required`, dan `example` untuk `category`. Contoh respons yang ditulis terpisah pada endpoint masih dapat memuat `category` selama eksperimen; pembuktian drift berfokus pada skema Product.

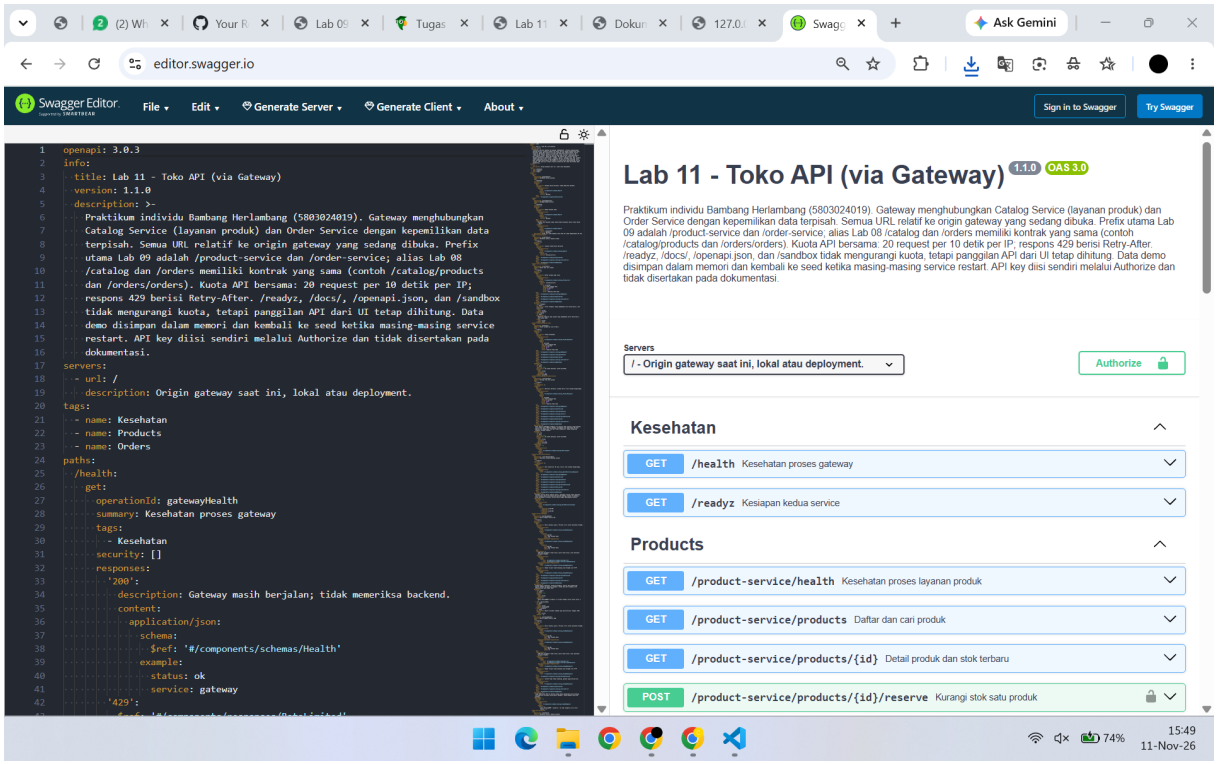
OpenAPI pada dasarnya mengizinkan properti tambahan sehingga validasi biasa belum tentu menolak field respons yang belum didokumentasikan. Pencegahannya meliputi tinjauan perubahan skema, pengujian kelengkapan field/kontrak pada CI, dan penggunaan sumber skema bersama. Eksperimen ini sengaja dibuat untuk praktikum, bukan temuan bug `category` pada kode awal.

5 Validasi OpenAPI dan impor Postman

Ekspor gateway disimpan sebagai `openapi.json` pada root setelah anotasi dipulihkan. Isinya sama secara struktural dengan `gateway/openapi.json`. Swagger Parser lokal menyatakan spesifikasi OpenAPI 3.0.3 valid.

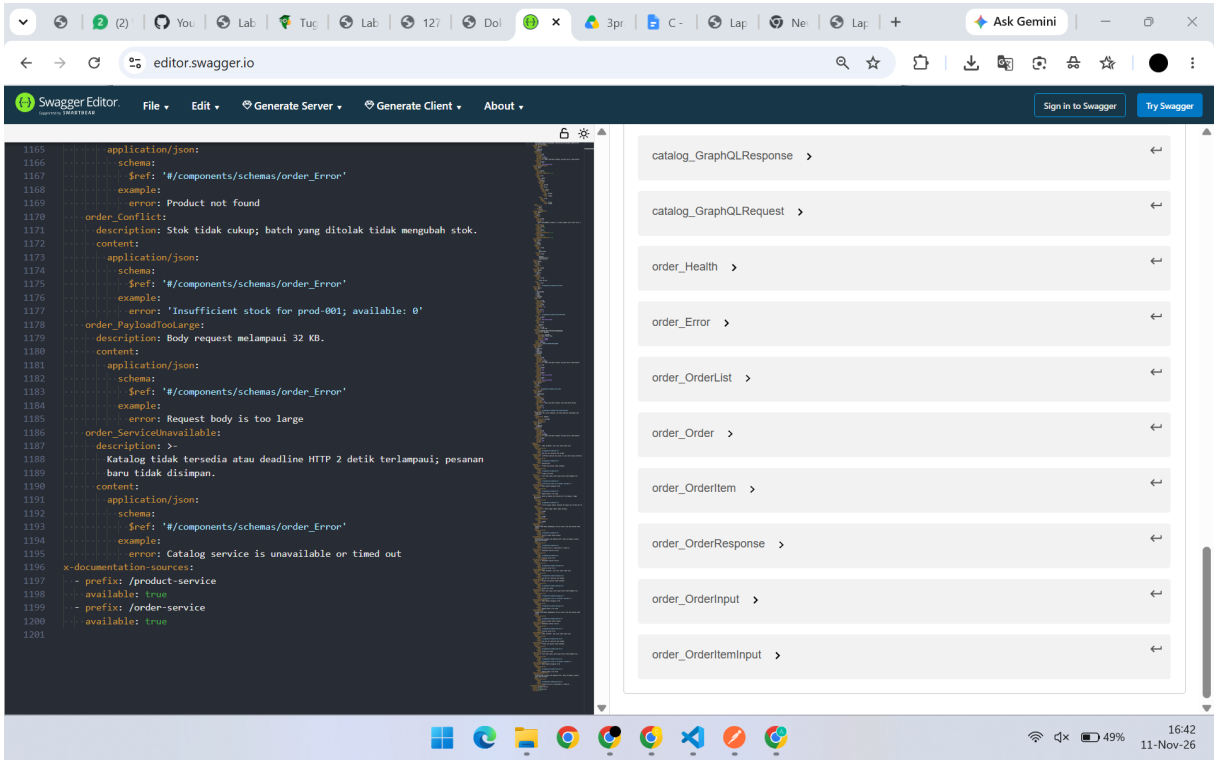
Isi file kemudian ditempel pada Swagger Editor, dan hasilnya terlihat pada Gambar 7. Panel kiri menampilkan sumber YAML: `openapi: 3.0.3`, `info` dengan judul Lab 11 - Toko API (via Gateway) dan versi 1.1.0, deskripsi gateway, `servers` berisi `/`, daftar `tags` (Kesehatan, Products, Orders), lalu `paths` yang dimulai dari `/health` dengan `operationId: gatewayHealth` serta respons 200 dan 429. Panel kanan merender spesifikasi yang sama sebagai Swagger UI, lengkap dengan lencana 1.1.0 dan OAS 3.0, tombol Authorize, pilihan server `/` (origin gateway saat ini), grup Kesehatan (`/health`

dan `/readyz`), dan grup Products (`/product-service/...`). Deskripsi pada kedua panel menyebut kuota API 20 request per 10 detik per IP dan bahwa API key diisi sendiri lewat Authorize, bukan disertakan pada dokumentasi. Karena editor dapat mengurai YAML dan merender endpoint tanpa tanda error pada bagian awal ini, hasilnya mendukung temuan Swagger Parser bahwa spesifikasi valid. Bagian akhir dokumen diperiksa pada Gambar 8.



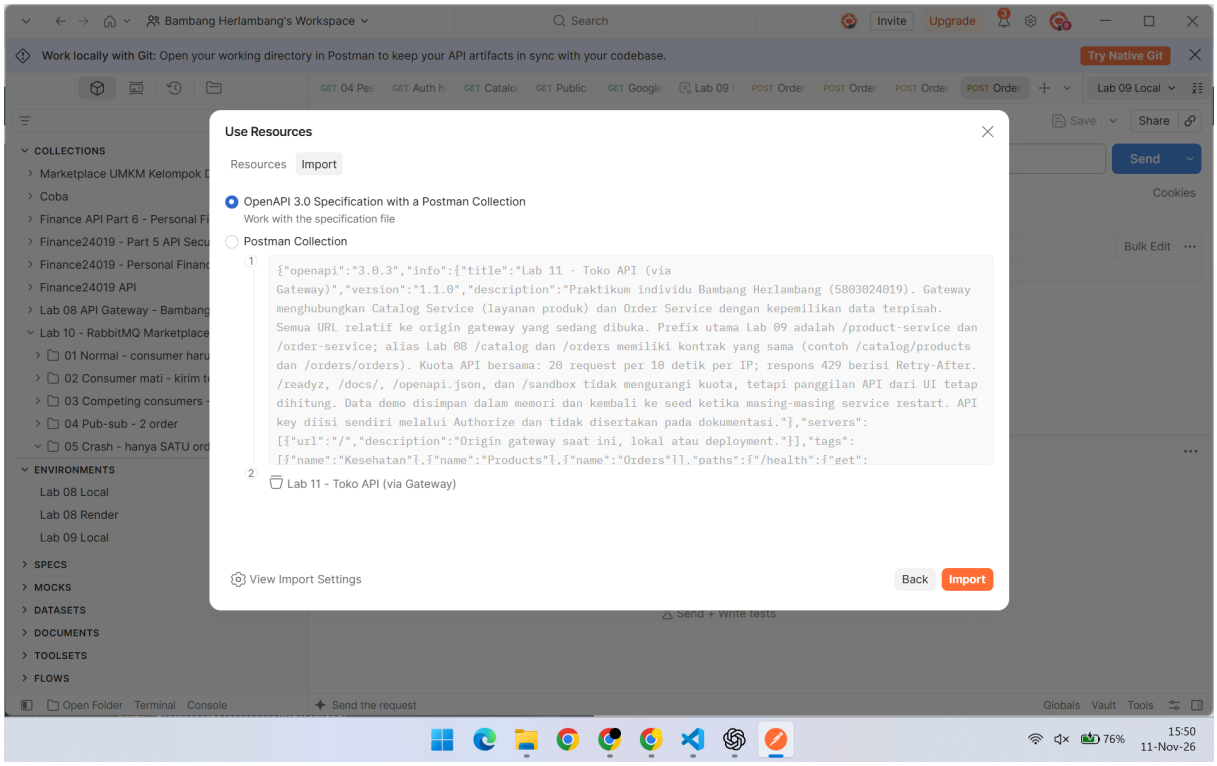
Gambar 7. Impor dan tampilan spesifikasi pada Swagger Editor. Sumber: Screenshot (1985).png.

Gambar 8 diambil setelah editor digulir sampai akhir dokumen, yaitu baris 1201. Panel kiri menampilkan bagian akhir YAML: respons bersama pada `components` (`order_Conflict` dengan deskripsi stok tidak cukup dan contoh `available: 0`, `order_PayloadTooLarge` untuk body di atas 32 KB, dan `order_ServiceUnavailable` untuk katalog yang tidak tersedia atau melewati deadline HTTP 2 detik), lalu ekstensi `x-documentation-sources` yang mencatat kedua sumber, yaitu `/product-service` dan `/order-service`, dengan `available: true`. Panel kanan, yang ikut tergulir ke bagian Schemas, menampilkan skema bernamespace `catalog_` dan `order_` (antara lain `order_Order`, `order_OrderInput`, dan `order_OrderResponse`) yang semuanya berhasil dirender. Tidak ada tanda error pada bagian yang tampak. Bersama Gambar 7, yang menampilkan awal dokumen, kedua gambar mencakup dua ujung spesifikasi, dan seluruh dokumen sudah digulir dari atas sampai bawah tanpa ditemukan error, sesuai ketentuan praktikum (peringatan boleh, error tidak boleh). Status `available: true` pada kedua sumber juga menegaskan bahwa spesifikasi ini diambil saat kedua service sehat.



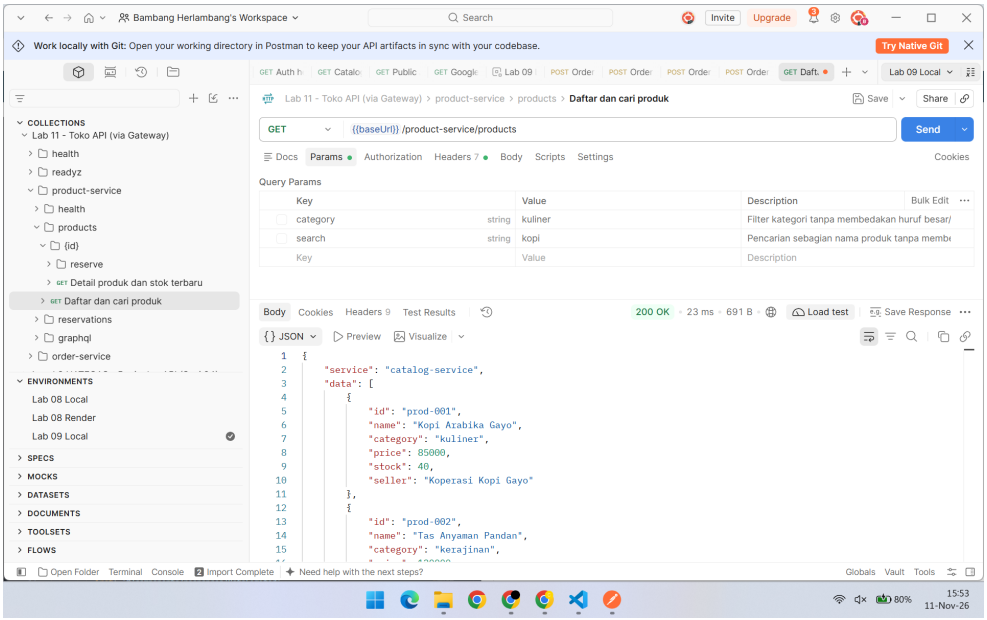
Gambar 8. Bagian akhir spesifikasi pada Swagger Editor (baris 1165 sampai 1201) tanpa tanda error. Sumber: tangkapan layar pengguna, 11 November 2026 pukul 16:42.

Gambar 9 adalah dialog impor Postman (Use Resources, tab Import). Dua opsi tersedia, yaitu OpenAPI 3.0 Specification with a Postman Collection dan Postman Collection. Opsi pertama terpilih dengan keterangan Work with the specification file, dan kotak pratinjau menampilkan awal isi `openapi.json` (`"openapi":"3.0.3"`, judul Lab 11 - Toko API (via Gateway), versi 1.1.0). Di bawahnya tertulis nama koleksi yang akan dibuat, Lab 11 - Toko API (via Gateway). Dengan demikian Postman mengenali file sebagai OpenAPI 3.0 dan membuat koleksi langsung dari spesifikasi, bukan dari koleksi bawaan, lalu impor dijalankan dengan tombol Import.



Gambar 9. Dialog impor file OpenAPI langsung ke Postman. Sumber: Screenshot (1986).png.

Gambar 10 memperlihatkan hasil setelah impor. Panel kiri berisi koleksi Lab 11 - Toko API (via Gateway) dengan folder yang mengikuti path spesifikasi, yaitu health, readyz, product-service (health, products dengan subfolder `{id}` berisi reserve dan Detail produk dan stok terbaru, Daftar dan cari produk, reservations, dan graphql), serta order-service. Seluruh struktur ini terbentuk otomatis tanpa membuat request satu per satu. Request yang dibuka adalah `GET {{baseUrl}}/product-service/products` (Daftar dan cari produk). Parameter `category` (kuliner) dan `search` (kopi) tercantum pada Query Params beserta deskripsi yang diambil dari spesifikasi, tetapi dinonaktifkan sehingga request mengambil semua produk. Setelah Send, Postman menampilkan 200 OK dengan waktu 23 ms dan ukuran 691 B. Isi JSON-nya berupa `service: catalog-service` dan daftar `data`, termasuk `prod-001` (kuliner) dan `prod-002` Tas Anyaman Pandan (kerajinan), yang semuanya memuat `category`. Hasil ini membuktikan bahwa spesifikasi dari gateway dapat dipakai ulang oleh tool lain dan sejalan dengan skema Product yang sudah diperbaiki.



Gambar 10. Hasil aktual request Postman sesudah impor OpenAPI. Sumber: Screenshot (1990).png.

Koleksi JSON hasil konverter tersedia pada folder `docs/postman/` sebagai `lab11.postman_collection.json`. Bukti manual berasal dari impor OpenAPI langsung. Status impor yang belum diverifikasi pada rekaman demo otomatis dipertahankan sebagai riwayat saat demo dijalankan; status manual terbaru dan hash screenshot tersimpan dalam `verification.json` pada folder `docs/evidence/lab11/manual/`.

6 Penerapan ke proyek: Auth + Gateway

Komponen individual Bambang Herlambang (5803024019) pada proyek Marketplace UMKM Lokal adalah **Auth + Gateway**. Untuk penerapan Lab 11, tiga endpoint Auth yang sudah tersedia dalam kode proyek didokumentasikan secara khusus: POST `/auth/token`, GET `/.well-known/jwks.json`, dan GET `/auth/me`.

Spesifikasi mandiri disimpan sebagai `docs/proyek-auth-service.openapi.yaml`. Dokumen ini melengkapi kontrak praktikum toko; root `openapi.json` tetap berisi dokumentasi produk dan pesanan. Skema masukan, respons sukses, error, autentikasi, serta peran gateway dijelaskan pada subbagian berikut.

Kontrak ketiga endpoint cocok dengan versi proyek pada commit `4a69c54`. Sumber kode lokal juga diperiksa; manifest sumber terdapat pada `docs/lab11/proyek-auth-source.json`. Dokumentasi ini tidak menyatakan seluruh perubahan lokal proyek sudah dipublikasikan atau deployment produksi telah diperbarui.

6.1 Penerbitan token layanan: POST /auth/token

Endpoint ini menerbitkan JWT untuk client layanan yang terdaftar. Masukan memakai `application/x-www-form-urlencoded`, bukan JSON atau login email/password pengguna. Login pengguna tetap menggunakan Google OIDC. Skema request bernama `ClientCredentialsRequest`.

Field	Tipe	Aturan masukan
`grant_type`	string	Wajib; nilai `client_credentials`.
`client_id`	string	Wajib; client terdaftar pada registry server.
`client_secret`	string	Wajib; diverifikasi terhadap hash secret server.
`scope`	string	Opsional, panjang 1–1000; default `read:catalog`; daftar scope dipisahkan spasi.

Scope harus dikenal dan diizinkan untuk client. Scope reservasi hanya boleh diterbitkan untuk order-service yang memiliki izin eksplisit; token tidak otomatis memperoleh seluruh scope.

Contoh masukan berikut menggunakan placeholder, bukan secret produksi:

```
grant_type=client_credentials
client_id=order-service
client_secret=GANTI_DENGAN_SECRET_CLIENT_LOKAL
scope=read:catalog
```

Respons sukses **200** memakai `ClientTokenResponse`; seluruh field berikut wajib:

Field	Tipe	Makna
`access_token`	string	JWT aplikasi RS256 dengan role service.
`token_type`	string	Nilai `Bearer`.
`expires_in`	integer	Nilai 900 detik.
`scope`	string	Scope yang diminta dan diizinkan, tanpa duplikasi.

Kode error: 400 untuk parameter berulang (`INVALID_REQUEST`), grant tidak didukung (`UNSUPPORTED_GRANT_TYPE`), atau scope tidak diizinkan (`INVALID_SCOPE`); 401 `INVALID_CLIENT` untuk client/secret salah; 413 `BODY_TOO_LARGE` untuk body di atas 4096 byte; 415 `UNSUPPORTED_MEDIA_TYPE` untuk Content-Type selain form; serta 500 `INTERNAL_ERROR` untuk kegagalan internal. Semua memakai skema error pada Bagian 6.4.

6.2 Kunci publik JWT: GET /.well-known/jwks.json

Endpoint ini tidak memerlukan body, query, atau bearer token. Header opsional `X-Correlation-ID` berupa UUID dapat dikirim untuk penelusuran request. Respons **200** memakai skema `PublicJwks`, berupa objek dengan field wajib `keys` (array objek JWK).

Field setiap JWK	Tipe	Nilai/makna; seluruhnya wajib
`kty`	string	`RSA`.
`kid`	string	Identifier public signing key.
`n`, `e`	string	Modulus dan exponent RSA publik.
`alg`	string	`RS256`.
`use`	string	`sig`.

Parameter RSA privat seperti d, p, dan q tidak dikembalikan. Kegagalan membaca konfigurasi public key menghasilkan **500 INTERNAL_ERROR**. Path proyek yang benar adalah `/.well-known/jwks.json`; gateway memakainya untuk memperoleh public key.

6.3 Profil akun sendiri: GET /auth/me

Masukan berupa header `Authorization:Bearer<JWT_PENGGUNA>` dan X-Correlation-ID opsional; tidak ada request body. JWT harus valid, role buyer/seller/admin, dan memiliki scope `read:catalog`. Akun harus ACTIVE pada database Auth. Token role service ditolak. Query tambahan, misalnya userId, tidak diterima; identitas selalu berasal dari JWT terverifikasi.

Respons **200** memakai `OwnProfile`. Properti tambahan tidak diizinkan, dan seluruh field berikut wajib:

Field	Tipe	Isi
`id`	string	ID internal pemilik token.
`email`	string/email	Alamat email akun.
`displayName`	string	Nama tampilan akun.
`role`	string	buyer, seller, atau admin.
`status`	string	ACTIVE.

Google subject, password, client secret, dan private key tidak dikembalikan. **Error:** 400 BAD_REQUEST untuk query tidak didukung; 401 UNAUTHENTICATED untuk token tidak ada/invalid/expired; 403 FORBIDDEN untuk role/scope salah atau akun tidak aktif/tidak ditemukan; 503 SERVICE_UNAVAILABLE bila mode postgres atau penyimpanan Auth belum tersedia; serta penanganan global 500 INTERNAL_ERROR.

6.4 Skema error, header, dan routing gateway

Semua respons error Auth menggunakan `ErrorResponse`. Field `error` wajib berupa objek; code, message, dan correlationId di dalamnya juga wajib. Ketiganya bertipe string, dengan format UUID pada correlationId. Contoh ilustratif:

```
{
  "error": {
    "code": "UNAUTHENTICATED",
    "message": "Token tidak valid atau kedaluwarsa",
    "correlationId": "00000000-0000-4000-8000-000000000002"
  }
}
```

Respons diberi `Cache-Control:no-store` dan `X-Correlation-ID`. UUID request yang valid diteruskan; nilai tidak valid diganti UUID baru. Contoh kontrak tidak berisi token aktif atau secret nyata.

Auth proyek lokal berjalan pada **3001**; gateway proyek pada **3000**. Ketiga path diteruskan tanpa prefix product-service/order-service. Port ini berbeda dari gateway praktikum toko (8080). File YAML mendeskripsikan **Auth langsung**; lewat gateway dapat muncul tambahan error berikut:

HTTP	Penyebab di lapisan gateway
401 / 405	Token yang dikirim invalid; atau metode tidak diizinkan (header Allow).
413 / 415	Request di atas 64 KiB; atau encoding request tidak didukung. Limit Auth token tetap lebih ketat, 4096 byte.
429	Kuota default 20 request per 10 detik habis; Retry-After menyatakan waktu tunggu.
502	Gangguan upstream atau respons di atas 1 MiB.
503	Binding, limiter, atau verifikasi JWKS belum tersedia.
504	Deadline upstream gateway terlampaui; default 5 detik.

6.5 Hasil verifikasi dan batas penerapan

Spesifikasi Auth lolos Swagger Parser OpenAPI 3.0.3; referensi dapat di-resolve dan **13 contoh request/response** cocok dengan skemanya. **Empat tes proyek terpilih lulus**: JWKS publik tanpa parameter RSA privat, validasi client credentials, akses profil sendiri/penolakan akun suspended, serta penerusan profil oleh gateway. Tes memakai key dan penyimpanan uji terisolasi, bukan request produksi.

Dokumentasi ini melengkapi syarat minimal tiga endpoint untuk komponen Auth milik Bambang. Tidak ada login Google, token produksi, migrasi, atau deployment baru dalam pekerjaan dokumentasi ini. Penerapan anggota lain dan kesiapan checkout/payment end-to-end tidak disimpulkan dari hasil ini. Dokumen pendamping berada di `docs/lab11/penerapan-proyek-auth-gateway.md`.

7 Refleksi

Pendekatan code-first memudahkan pengembang menjaga dokumentasi dekat dengan handler dan menghasilkan Swagger UI dari kode yang sudah berjalan. Dibanding contract-first pada Lab 03, pendekatan ini lebih cepat diterapkan pada service lama, tetapi kontrak baru diketahui setelah implementasi dan anotasi masih dapat tertinggal. Eksperimen category menunjukkan bahwa generator membaca anotasi, bukan memahami seluruh perubahan objek respons, sehingga code-first mengurangi tetapi tidak menghilangkan drift. Tim dapat mencegah drift melalui tinjauan perubahan skema, pengujian kontrak pada CI, pemeriksaan field respons, dan aturan bahwa perubahan endpoint harus disertai perubahan dokumentasi. Event `order.created` dari Lab 10 sebaiknya didokumentasikan dengan AsyncAPI yang menjelaskan broker RabbitMQ, exchange/channel, payload, producer, consumer, serta perilaku ack/redelivery, karena OpenAPI mendeskripsikan kontrak HTTP dan tidak mencakup alur event tersebut.

Sumber dan lokasi bukti

Panduan: `https://webservice.philipus.my.id/praktikum/lab-11`. Laporan Markdown: `docs/lab11/README.md`. Bukti utama: `docs/evidence/lab11/manual/`. Semua screenshot pada laporan ini adalah PNG asli pengguna pada 11 November 2026, ditampilkan tanpa perubahan isi; gambar yang menampilkan API key tidak dimasukkan.

Spesifikasi penerapan individual: `docs/proyek-auth-service.openapi.yaml`. Penjelasan dan batas verifikasi: `docs/lab11/penerapan-proyek-auth-gateway.md`. Manifest sumber Auth: `docs/lab11/proyek-auth-source.json`.